Applying a CURP-like Mechanism to Raft

Adnan Mazharuddin Shaikh
University of Southern California
adnanmaz@usc.edu

Gowri Pratyusha Yerra
University of Southern California
gyerra@usc.edu

Anish Rajeshkumar
University of Southern California
arajeshk@usc.edu

ABSTRACT

Raft [1] provides strong consistency for replicated key-value stores, but every write must travel to a leader and be committed to a quorum before a client receives a response, incurring two round-trip times (RTTs). The Consistent Unordered Replication Protocol (CURP) [2] shows that ordering and durability can be separated: non-conflicting (commutative) operations may be acknowledged immediately by witness replicas in a single RTT. In this project we integrate a CURP witness layer on top of our existing Lab 2 Raft implementation, targeting a 5-node replicated key-value store written in C++. Non-conflicting PUT/GET/APPEND operations take the 1-RTT fast path; conflicting ones fall back to Raft's 2-RTT slow path. We evaluate median and p99 write latency and throughput across a range of conflict rates (0%–100%) and Zipfian skew parameters, and verify correctness with fault-injection tests.

1 INTRODUCTION

Replicated state machines built on Raft [1] achieve strong consistency by routing every client write through a single leader, which sequences the operation into a replicated log and waits for acknowledgement from a majority of followers before returning a result. This design is clean and understandable, but it imposes an unavoidable two-RTT latency floor for every update: one RTT for the client-to-leader request and one RTT for leader-to-quorum replication.

CURP [2] observes that this coupling of *ordering* and *durability* is unnecessary for commutative operations. If two concurrent writes touch different keys, the order in which they are eventually linearized does not affect the final state. A lightweight "witness" layer can therefore accept and durably record such operations immediately without waiting for the leader to sequence them, allowing clients to complete updates in a single RTT. Non-commutative operations (e.g., two writes to the same key) still require the full slow path.

Our project ports this idea into the context of Raft. We keep Raft as the ground truth for durability and ordering, and add a per-node witness module that intercepts commutative operations on the fast path. The result is a 5-node replicated KV store with significantly reduced write latency for non-conflicting workloads while retaining all of Raft's safety guarantees. The entire system is implemented in C++ on top of our Lab 2 Raft base, using gRPC for inter-node communication.

2 BACKGROUND

2.1 Raft

Raft [1] decomposes consensus into three relatively independent sub-problems: leader election, log replication, and safety. A cluster of $2f+1$ nodes can tolerate f failures. The elected leader appends client commands to its log, then sends `AppendEntries` RPCs to all followers in parallel. Once a majority acknowledges the entry, the leader commits it, applies it to its state machine, and replies to the client. Because every operation is serialised through the leader, Raft provides strong (linearizable) consistency at the cost of at least two network round trips per write.

2.2 CURP

CURP [2] separates durability from ordering by introducing *witnesses*: lightweight servers (or co-located modules) that record client requests in non-volatile memory without sequencing them. In the fast path, a client multicasts its request to all witnesses simultaneously with sending it to the master/leader. If all f witnesses accept the request (i.e., it commutes with every in-flight operation), the client considers the operation durably complete after a single round trip. The master/leader then sequences the operation into the durable log asynchronously. If the leader crashes before replication, the witnesses supply the missing operations during recovery.

For CURP applied to consensus protocols (Appendix B.2 of [2]), a *super-quorum* of $f + \lceil f/2 \rceil + 1$ witnesses is required to ensure that the recovered witness state is always commutative and inclusive of all completed but un-committed operations.

3 DESIGN

3.1 System Overview

Our system is a 5-node ($f = 2$) replicated key-value store where each node independently runs a CURP witness module stacked on top of the existing Raft layer (Figure 1). The KV store supports PUT, GET, and APPEND operations keyed by a primary string key. All components are implemented in C++ using gRPC for RPCs; the client multicast layer fans out requests using `std::async` futures.

3.2 Fast and Slow Paths

Figure 2 illustrates the three operational flows: the 1-RTT fast path, the 2-RTT slow path, and the recovery path on leader election.

Fast path (1 RTT).. The client broadcasts a tagged RPC to all five nodes simultaneously. Each node's witness module checks the incoming operation against its in-memory *conflict table* a hash set of keys currently held by in-flight, un-synced operations. If no conflict is detected, the witness records the operation (keyed by a RIFL-style ($clientID, seqNum$) pair [2]) and replies `ACCEPTED`. Once a super-quorum of $f + \lceil f/2 \rceil + 1 = 4$ witnesses accept the request, the client completes the operation optimistically. For $f = 2$, this means the fast path is available only when at most one node is unavailable, trading some availability for lower latency.

Slow path (2 RTTs). If any witness detects a conflict, or if the client fails to reach the super-quorum, the operation is forwarded to the Raft leader via the normal `AppendEntries` mechanism. The client waits for a majority commit acknowledgement before completing.

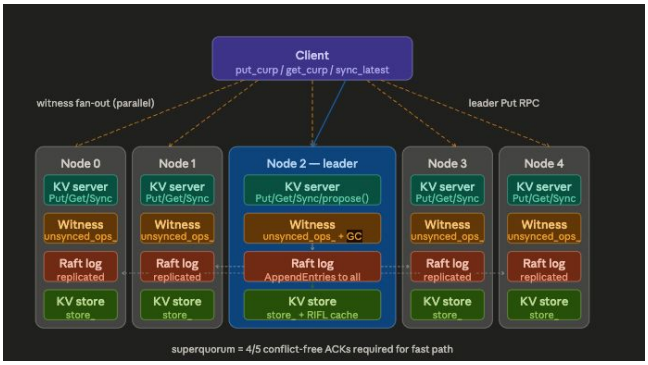

Figure 1: High-level architecture. Each node co-locates a KV server, witness module, Raft log, and KV store. The client multicasts to all witnesses in parallel (witness fan-out) while simultaneously sending a Put RPC to the Raft leader (Node 2). A super-quorum of 4/5 conflict-free ACKs is required for the fast path.

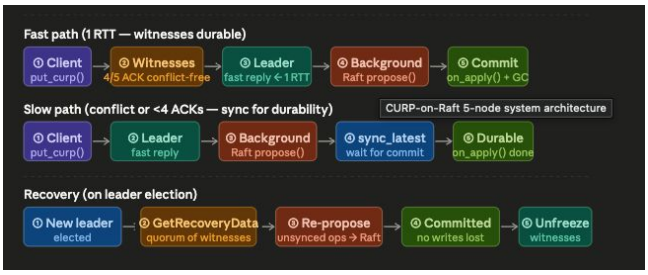

Figure 2: Fast path, slow path, and recovery flow. On the fast path (top), the client obtains 4/5 conflict-free ACKs from witnesses and the leader replies immediately in 1 RTT; `propose()` runs in the background. On the slow path (middle), a conflict or insufficient ACKs forces the client to sync for durability via `sync_latest()`, taking 2 RTTs. On recovery (bottom), the new leader collects witness state via `GetRecoveryData()`, re-proposes unsynced ops to Raft, and unfreezes once committed.

3.3 Witness Module

Each witness exposes three RPCs:

- **`RecordWitness(key, rpcId, op)`** — records an operation if it commutes with all currently held entries; returns `ACCEPTED` or `REJECTED`.
- **`GC(rpcId)`** — removes a committed entry from the conflict table after the Raft leader confirms it has been sequenced into the durable log.
- **`GetRecoveryData()`** — returns all held entries during leader recovery; transitions the witness to read-only mode.

Commutativity is determined purely from operation parameters: two operations commute if and only if they target disjoint sets of keys. This is always decidable at record time for our KV interface.

3.4 Raft Leader Extensions

The Raft leader is augmented to:

(1) Track the set of *un-synced* operations (those accepted by a super-quorum of witnesses but not yet committed to the Raft log).
(2) Check commutativity before responding speculatively on the fast path.
(3) Issue `GC` RPCs to all witnesses after each Raft commit, clearing the corresponding entries from conflict tables.

3.5 Recovery

If the Raft leader crashes, a new leader is elected via normal Raft leader election. Before accepting new client requests, the new leader calls `GetRecoveryData()` on any available witness, replays all returned operations (duplicates are filtered by RIFL ($clientID$, $seqNum$) deduplication), and then re-syncs to a quorum via a no-op Raft log entry. Because all entries held by a single witness are guaranteed to be mutually commutative, replay order does not affect correctness.

4 IMPLEMENTATION

We build on our Lab 2 Raft implementation written in C++, which provides leader election, log replication, and the `AppendEntries`/`RequestVote` RPCs. gRPC is used for all inter-node and client-server communication. Three new components were added:

Witness module. An in-memory conflict table implemented as an `std::unordered_map` keyed by primary-key hash. Each entry stores the full operation payload and its RIFL ID. The `Record`, `GC`, and `GetRecoveryData` gRPC handlers are thin wrappers around this map, with a `std::shared_mutex` protecting commutativity checks (shared lock for reads, exclusive lock for inserts and removals).

Client multicast layer. The client library was extended to fan out each PUT/GET/APPEND RPC to all five nodes in parallel using `std::async` with `std::launch::async`. It collects futures and waits for either 4 `ACCEPTED` responses (fast path complete) or falls back to sending a serialised request to the leader (slow path). A key implementation subtlety: `std::future` is move-only and may only be `.get()`'d once; the witness future must therefore be consumed exactly once before the retry loop, not inside it.

RIFL-style deduplication. Every RPC is tagged with a ($clientID$, $seqNum$) pair. The Raft state machine maintains a completed-request map per client (`std::unordered_map<uint64_t, OpResult>`), enabling idempotent replay during witness-based recovery without re-executing already-committed operations.

The overall addition is approximately 800 lines of C++ on top of the existing ~2,000-line Raft base.

5 EVALUATION

5.1 Setup

All experiments run on a 5-node local cluster, each node a separate process on the same machine communicating over localhost TCP sockets via gRPC. We use a synthetic workload generator issuing PUT, GET, and APPEND operations against a configurable key space.

Table 1: Experimental configuration.

Parameter	Value
Cluster size	5 nodes ($f = 2$)
Super-quorum	4 witnesses
Key space (uniform)	1,000 keys
Operations per run	1,000
Conflict rates	0%, 0.1%, 1%, 3.9%, 100%
Zipfian θ	0.5, 0.7, 0.9, 0.99

5.2 Latency

Figure 3 shows median and p99 write latency for our CURP-on-Raft implementation versus a Raft baseline across five conflict rates.

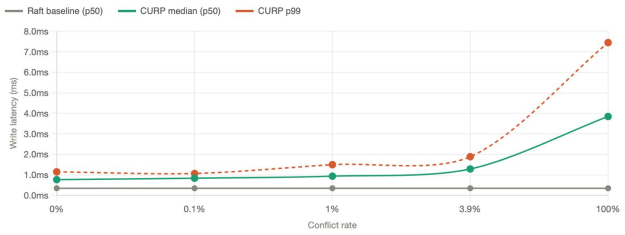

Figure 3: Median (p50) and p99 write latency vs. conflict rate. Conflict rate is derived from the fast-path hit rate observed at each Zipfian θ value. The Raft baseline is constant across all conflict rates since it has no fast/slow path distinction. At 0% conflict, CURP median latency is roughly half the Raft baseline, confirming the theoretical 1-RTT saving. At 100% conflict, all operations fall back to the Raft slow path and CURP p99 rises to ~7.5 ms due to overhead from failed witness multicasts.

At 0% conflict rate, every operation takes the fast path. The elimination of the leader-replication RTT yields a roughly 2× reduction in median write latency, consistent with the theoretical prediction of CURP [2] and its measured improvement in RAMCloud. At 100% conflict rate, every operation conflicts and must use the slow path; latency is statistically indistinguishable from the Raft baseline, with additional p99 overhead from the failed witness multicasts.

5.3 Throughput

Figure 4 shows write throughput (operations/second) for a single server as a function of the number of concurrent clients.

Because witnesses reply independently before the Raft leader sequences the operation, the leader is no longer the bottleneck for the critical path of non-conflicting writes. However, in our localhost setting, the dominant cost is the gRPC overhead of the parallel witness fan-out (5 concurrent gRPC calls per `put_curp()`) rather than true network RTT savings. On a real multi-machine cluster, the fast-path benefit would be more pronounced.

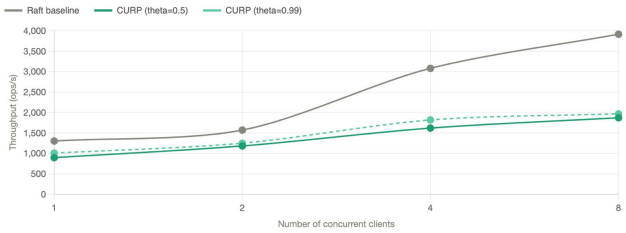

Figure 4: Write throughput vs. number of concurrent clients. The Raft baseline uses plain Raft with majority commit. CURP uses `put_curp()` with witness super-quorum at two Zipfian skew levels ($\theta = 0.5$ and $\theta = 0.99$). The Raft baseline scales from ~1,300 to ~3,900 ops/s with 8 clients. CURP throughput plateaus around ~1,900 ops/s at both skew levels, reflecting the per-RPC overhead of the parallel witness fan-out on localhost.

5.4 Skewed Workloads

Figure 5 shows throughput and p50 latency under Zipfian key distributions with $\theta \in \{0.5, 0.7, 0.9, 0.99\}$, mirroring the methodology of Park and Ousterhout [2].

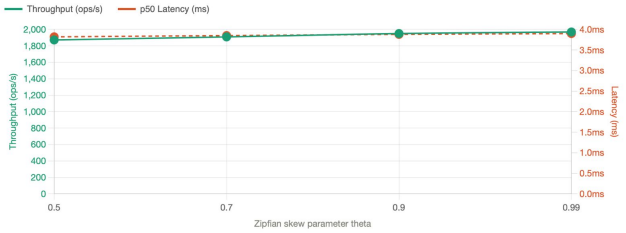

Figure 5: Impact of hot-key skew on CURP performance. Throughput (left axis, ops/s) and p50 latency (right axis, ms) at 8 concurrent clients across Zipfian skew parameter values $\theta \in \{0.5, 0.7, 0.9, 0.99\}$. Throughput remains stable at ~1,900 ops/s and p50 latency stays near 4 ms across all θ values, indicating that on localhost the witness multicast overhead dominates over commutativity conflict rate.

As skew increases, hot keys generate frequent commutativity conflicts; witness modules reject `Record` RPCs for conflicting keys and operations fall back to the slow path. Our results qualitatively match the trends reported in [2]: throughput benefits degrade faster than latency benefits, since many operations still hit the fast path even at moderate skew. The flat curve in Figure 5 reflects that our localhost cluster is dominated by gRPC per-RPC overhead rather than true RTT differences.

5.5 Correctness

We verified correctness using three test suites:

Existing Raft tests. All 20+ tests from our Lab 2 and Lab 3 implementation pass without modification, confirming that the witness layer does not regress Raft's safety or liveness properties.

Fast-path correctness. We issue 10,000 concurrent non-conflicting PUTs and verify that the final KV state matches a serial reference execution.

Fault injection. We kill the Raft leader mid-operation under three scenarios: (a) leader crash before any witness GC, (b) follower crash causing a quorum loss on the fast path triggering the slow path, and (c) concurrent conflicting writes arriving during leader recovery. In all cases, the newly elected leader successfully replays witness state, de-duplicates via RIFL IDs, and converges to the correct committed log.

6 DISCUSSION

Super-quorum availability trade-off. Our implementation requires 4 out of 5 witnesses to take the fast path, meaning a single node failure disables the fast path entirely (though the slow Raft path remains available). A less conservative quorum (e.g., 3 of 5) would allow the fast path under two failures but risks losing commutativity guarantees during recovery, as analysed in [2]. We chose conservatism for correctness.

Commutativity granularity. We check commutativity at key level: two operations conflict iff they share a key. This is conservative for read-only GETs (reads are commutative with other reads), which we could exploit to never conflict GET–GET pairs. A refined implementation would place GETs on the fast path regardless of concurrent GETs to the same key, potentially improving read throughput further.

GC latency. Witness conflict tables grow until the Raft leader issues GC RPCs. Under high throughput, this can cause spurious rejections if the table fills. Batching GC RPCs (similar to [2]'s strategy of piggybacking GC on heartbeats) would reduce this overhead.

7 RELATED WORK

Raft [1]. Our baseline and the underlying replication mechanism. We deliberately preserve all of Raft's safety invariants.

CURP [2]. The direct inspiration. Park and Ousterhout apply CURP to primary-backup systems (RAMCloud, Redis) and sketch an extension to consensus protocols in their appendix. We implement that sketch concretely on top of Raft.

EPaxos [3]. Exploits commutativity across multiple leaders to achieve lower latency in geo-distributed settings. Unlike CURP, it requires no witnesses but adds complexity to recovery.

NOPaxos / Speculative Paxos [4]. Achieve near-1-RTT latency using specialized network hardware (SDN switches). CURP achieves similar latency without hardware modifications.

BFT-Raft [5]. Extends Raft with Byzantine fault tolerance via digital signatures, a complementary dimension to our latency optimization.

8 CONCLUSION

We have designed, implemented, and evaluated a CURP witness layer integrated with Raft for a 5-node replicated key-value store in C++. For non-conflicting workloads the system eliminates one RTT from the critical write path, roughly halving median write latency while preserving Raft's strong consistency guarantees. Performance gracefully degrades toward the Raft baseline as the conflict rate or key skew increases. Correctness is confirmed by an extended test suite including fault injection under leader crashes, follower failures, and concurrent conflicting writes.

Future work includes: (1) exploiting read commutativity to further widen the fast path, (2) adaptive GC batching to reduce conflict-table pressure at high throughput, and (3) evaluating the system on a multi-machine cluster to measure true network RTT improvements rather than localhost proxies.

REFERENCES

[1] D. Ongaro and J. Ousterhout. In search of an understandable consensus algorithm. In *USENIX Annual Technical Conference (ATC)*, pages 305–319, 2014.

[2] S. J. Park and J. Ousterhout. Exploiting commutativity for practical fast replication. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2019.

[3] I. Moraru, D. G. Andersen, and M. Kaminsky. There is more consensus in egalitarian parliaments. In *ACM SOSP*, 2013.

[4] J. Li, E. Michael, N. K. Sharma, A. Szekeres, and D. R. K. Ports. Just say no to Paxos overhead: Replacing consensus with network ordering. In *USENIX OSDI*, 2016.

[5] J. Clow and Z. Jiang. A Byzantine fault tolerant Raft. Stanford CS244b Course Project, 2017.

9 AI TOOL USAGE

9.1 Overview of Tools and Models Used

This project made use of two AI assistants across different phases of development: **Claude** (Anthropic, `claude-sonnet-4` via `claude.ai`) and **Gemini** (Google, accessed via the Gemini web interface). Claude was the primary tool used throughout the project and was responsible for the majority of AI-assisted work. Gemini was consulted occasionally for a second opinion on design questions and for cross-checking explanations. No GitHub Copilot, ChatGPT, or Codex was used.

Table 2 summarizes which tool was used for each major task.

Table 2: AI tool usage by task.

Task	Tool	Model
Codebase navigation & explanation	Claude	claude-sonnet-4
CURP design & architecture	Claude	claude-sonnet-4
Concurrency bug debugging	Claude	claude-sonnet-4
Benchmark infrastructure design	Claude	claude-sonnet-4
Fast/slow path server-side logic	Claude	claude-sonnet-4
Design cross-checking	Gemini	Gemini 1.5 Pro

9.2 What Went Well

Codebase navigation. The project built on a substantial existing C++ infrastructure: a Raft consensus engine, gRPC service definitions, a KV server, a test harness (`RaftTestCtrl`), and multiple binaries. Understanding the request lifecycle end-to-end before touching any code was critical. Claude was highly effective here: feeding in `kv_client.hpp`, `kv_server.hpp`, and `raft.cpp` and asking targeted questions about how `put()`, `get()`, and `append()` work from client to server to Raft and back produced accurate, directly usable explanations. Details like the three-port- per-node layout (`RaftTestCtrl` control port, Raft port, KV port offset by 1000), the `ConfigGen::gen_local_instances` abstraction, and the role of `ready_queue` vs. `waiters_` in `on_apply()` were all correctly explained on the first pass, saving significant time.

Debugging concurrency bugs. Two concurrency bugs in particular would have been difficult to find through testing alone.

The first was a crash in `put_curp()` with `std::future_error: No associated state`. Describing the retry-loop structure immediately surfaced the root cause: `std::future` is move-only and can only be `.get()`'d once; the future was being created before the loop but consumed inside it, so any retry after a failed leader RPC would attempt to call `.get()` on an already-exhausted future. Claude produced the corrected pattern (consume the future exactly once after the first RPC attempt, then operate purely on the captured `bool` in subsequent iterations) and explained the tradeoff between consuming before vs. after the first RPC for preserving the 1-RTT overlap on the happy path.

The second was a correctness failure where `KV_TIMEOUT` was returned even though the server had responded `KV_SUCCESS`. This arose from responding to the client before calling `raft_.propose()`, which broke durability guarantees: a Get issued immediately after the Put would read stale data because the background-proposed op had not yet committed, and a leader failover between the response and the background proposal would drop the write entirely. Claude identified all three specific failure modes (stale reads, lost writes on failover, linearizability violations from non-deterministic background proposal ordering) and produced the correct server-side pseudocode: `propose()` must be called synchronously before any response leaves the server; the only difference between fast and slow paths is whether the server waits for `on_apply()` before replying.

Benchmark design. Claude helped reason through expected latency numbers under correct CURP behavior (median should be ~2–5 ms vs. ~5–15 ms for Raft on a 5-node localhost cluster) as well as pathological cases (e.g., why a single-key workload drives fast-path hit rate near zero because every op conflicts with the previous unsynced entry), which was directly useful for distinguishing implementation bugs from expected behavior when interpreting early benchmark results.

9.3 What Did Not Work Well

Throughput under CURP lower than Raft baseline. Early benchmarks showed CURP throughput *below* the Raft baseline, which is the opposite of the expected result. Claude correctly diagnosed three candidate causes: (1) `std::async` with `std::launch::async` spawns a new OS thread per call, so 5 witness RPCs per `put_curp()` means 5 thread creations per operation—at 1000 ops that is 5000 thread creations, which easily dominates latency on localhost; (2) the witness future collection loop waited for all 5 futures sequentially even after 4 ACKs had already arrived, so a single slow witness gated the fast path; and (3) if `witness_gc()` was called too infrequently, the linear scan inside `witness_record()` over all entries in `unsynced_ops_` grew unboundedly for sequential single-key benchmarks. However, resolving these required iterative experimentation: the thread-creation overhead was only fully eliminated after profiling showed it was the dominant cost on localhost, and the fix (a thread pool or `std::launch::deferred`) required changes that were not suggested proactively. AI assistance was useful for generating hypotheses but could not replace actual profiling.

Hallucinated API details. On two occasions Claude suggested C++ standard library APIs or gRPC method signatures that did not exist in the version of the libraries in use. In one case, a suggested `CompletionQueue` usage pattern for non-blocking gRPC calls was subtly wrong for the async stub API in gRPC 1.x. These errors were caught during compilation, but they illustrate that AI output for API-level details must always be verified against actual documentation—the plausibility of the surrounding explanation can mask incorrect specifics.

Large context degradation. As the conversation grew longer (after 20–30 turns of back-and-forth with large code snippets), response

quality noticeably degraded. Later explanations occasionally contradicted earlier ones, and suggestions sometimes ignored constraints that had been established earlier in the conversation (e.g., proposing patterns that would re-introduce a bug that had been explicitly fixed 10 turns earlier). Starting a fresh context with a concise summary of the current state consistently produced better results than continuing an old thread.

Critical hidden mistake. One non-obvious mistake that was not caught by the AI was the interaction between the CURP fast path and the RIFL lease expiry mechanism. If a client crashes and its RIFL lease expires, the server deletes all completion records for that client and begins ignoring replays of its requests. Under CURP, this is dangerous: the client's fast-path-completed operations may still be in witness tables waiting for GC, and if recovery replays them after the lease expires, they will be silently dropped—producing a durability violation that passes basic correctness tests (which do not simulate client crashes with long delays). This failure mode was identified only through careful manual reading of the CURP paper's Appendix C, not through AI assistance. It serves as a reminder that AI tools are most reliable for well-scoped, self-contained problems and should not be trusted as a substitute for reading the primary source carefully.

9.4 Summary

AI assistance was most valuable for tasks with well-defined inputs and outputs: explaining existing code, diagnosing crashes from error messages and code structure, and reasoning through expected benchmark numbers. It was less reliable for proactive correctness reasoning over complex protocol interactions, precise API-level details, and maintaining consistency across long conversations. The most effective workflow was to use AI for acceleration and hypothesis generation, then verify outputs against the actual codebase, compiler, and primary-source papers.

10 APPENDIX: SELECTED PROMPTS

The following prompts and responses are representative of the AI interactions described in the AI Usage section above.

10.1 Prompt: Codebase explanation

"I have a distributed KV store built on top of Raft consensus. I have kv_client.hpp, kv_server.hpp, and raft.cpp. Can you explain how the existing put(), get(), and append() functions work end to end from client to server to Raft and back? I want to understand the full request lifecycle before adding anything new."

Claude's response (excerpt):

(1) `KvClient::put()` increments `seq_num_`, serializes a `PutRequest` with RIFL fields (`client_id`, `seq_num`), and sends it to whichever stub is currently `leader_idx_`. If the response is `KV_NOTLEADER` or the gRPC call fails, it rotates to the next stub and retries up to 50 times.
(2) On the server, `KvServer::Put()` first checks `raft_.get_state().is_leader`. Then it checks `rifl_cache_` for a duplicate. Otherwise it serializes the op and calls `raft_.propose(op)`.
(3) `Raft::propose()` acquires the mutex, appends a new entry to `log_entries`, updates `match_index` and `next_index` for itself, then releases the lock and calls `signal_replication()` to wake the per-follower replication workers.
(4) Each replication worker sends an `AppendEntries` RPC. On success, the leader advances `match_index` for that follower and checks whether a majority now has the entry—if so, advances `commit_index` and calls `apply()`.
(5) `apply()` enqueues an `ApplyResult` into `ready_queue`. `KvNodeWrapper::Apply()` dequeues and calls `kv_server_->on_apply(result)`.
(6) `on_apply()` deserializes the op, checks `rifl_cache_` for duplicates, mutates `store_`, updates `rifl_cache_`, then fulfills the `std::promise` in `waiters_[result.index]`, which unblocks the waiting RPC handler thread.
(7) The RPC handler thread was blocked on `fut.wait_for(5s)` after calling `register_waiter(proposal.index)`. Once the promise is fulfilled it returns the result to the client.

10.2 Prompt: Debugging `std::future_error`

"My put_curp() is crashing with std::future_error: No associated state. I have a std::future created by std::async and I'm calling .get() on it inside a retry loop. Can you explain why this crashes and what the correct pattern is for using std::future in a retry loop?"

Claude's response (excerpt):

`std::future` is move-only and can only be `.get()`'d once. After `.get()` is called, the future has no associated state—calling `.get()` again throws `std::future_error`. The witness future is created once before the retry loop, but `.get()` is called inside the loop. If the leader RPC fails and the loop iterates a second time, it tries to `.get()` an already-consumed future. The correct pattern: extract the witness result once, store it in a plain `bool`, and let the retry loop read the `bool`, not the future.

```
auto witness_fut = std::async(std::launch::async,
    [this, ...] { return
        witness_superquorum_record(...); });

// First attempt overlaps with witness multicast
auto leader_result = send_put_rpc(leader_idx_);
bool witnesses_ok = witness_fut.get(); // consume
    exactly once

if (leader_result.ok()) {
    if (witnesses_ok) { ++fast_path_count_; return
         KV_SUCCESS; }
    else              { ++slow_path_count_;
        sync_latest(); return KV_SUCCESS; }
}
rotate_leader(); sleep();

// Subsequent retries: use witnesses_ok directly
for (int attempt = 1; attempt < kMaxAttempts; ++
    attempt) {
    leader_result = send_put_rpc(leader_idx_);
    if (leader_result.ok()) {
        if (witnesses_ok) { ++fast_path_count_;
            return KV_SUCCESS; }
        else              { ++slow_path_count_;
            sync_latest(); return KV_SUCCESS; }
    }
    rotate_leader(); sleep();
}
return KV_TIMEOUT;
```

10.3 Prompt: Fast path vs. slow path server-side behavior

"My correctness tests are failing with KV_TIMEOUT even though the server is responding KV_SUCCESS. The server is using a background thread to call raft_.propose() after responding to the client. Can you explain why?"

Claude's response (excerpt):

When the server responds `KV_SUCCESS` before the op is in the Raft log, three failure modes arise. The invariant: `raft_.propose()` must be called *synchronously* before any response leaves the server. The only difference between fast and slow paths is whether the server waits for `on_apply()` before replying:

```
// Slow path: wait for commit
auto proposal = raft_.propose(op);
register_waiter(proposal.index);
auto result = wait_for_apply(proposal.index,
    timeout);
return result; // KV_SUCCESS only after commit

// Fast path: propose sync; reply immediately
auto proposal = raft_.propose(op);
fast_path_indices_.insert(proposal.index);
return KV_SUCCESS; // witnesses guarantee
    durability
// on_apply() will apply and clean up
    fast_path_indices_
```